

RoboFusion Techathon 2026

Round 1 Submission

SMART CAMPUS SAFETY & RESPONSE GRID

An IoT-Based Multi-Sensor
Risk Assessment and Emergency Response System

TEAM NAME

GenZ Ignite

TEAM MEMBERS

Mst. Esat Jahan Akhi

Md. Muniruzzaman Bony

UNIVERSITY

University of Frontier Technology,
Bangladesh

SUBMISSION

System Documentation

July 2026

Table of Contents....

Executive Summary	3
Problem Statement	3
Proposed Solution	4
System Architecture	4
Data Flow	5
Circuit & Sensors	6
Backend Architecture	7
Database	8
Technology Stack	9
REST APIs	9
Risk Fusion Engine	11
Sensor Thresholds	11
Risk Weights	12
Override Rule 1 - Flame Emergency	13
Override Rule 2 - Combined Smoke and Gas Emergency	13
Override Rule 3 - Severe Flooding	14
Final Risk Score	14
Risk Level Classification	14
Results	15
Validation Table	18
Security	19
Limitations	19
Future Improvements	20
Conclusion	20
References	21

Executive Summary

The Smart Campus Safety & Response Grid is an IoT-based safety monitoring system designed to detect environmental hazards and provide real-time risk assessment.

The system collects sensor data from an ESP32-based zone node (simulated using Wokwi), securely transmits the readings to a FastAPI backend through REST APIs, stores the information in PostgreSQL, evaluates the situation using a weighted Risk Fusion Engine, and displays the results on a React dashboard.

The Risk Fusion Engine converts each sensor value into a normalized risk score, applies weighted fusion, and executes emergency override rules for critical conditions such as flame detection, combined smoke and gas emergencies, and severe flooding.

The developed prototype successfully demonstrates end-to-end communication from the IoT sensing layer to the visualization dashboard, providing a scalable architecture for future smart campus deployments.

Problem Statement

Educational institutions require continuous monitoring of environmental conditions to ensure student and staff safety.

Traditional monitoring approaches are largely manual and often fail to provide timely warnings for emergencies such as fire, gas leakage, flooding, abnormal vibration, or overcrowding.

The absence of centralized monitoring and intelligent risk assessment may delay emergency response and increase potential damage.

An integrated IoT platform capable of continuously collecting sensor data, evaluating risk, and presenting real-time information is therefore required.

Proposed Solution

The proposed Smart Campus Safety & Response Grid integrates IoT sensing, backend processing, database storage, and web visualization into a unified platform.

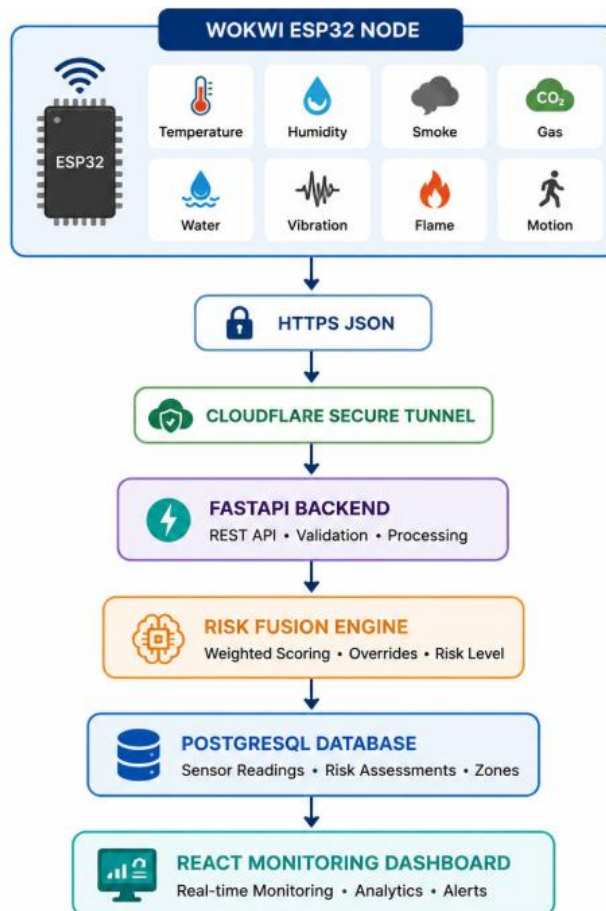
Environmental sensor readings are generated using an ESP32 node simulated in Wokwi. These readings are transmitted through HTTPS REST APIs to a FastAPI backend.

The backend stores sensor data in PostgreSQL and evaluates the overall safety condition using a weighted Risk Fusion Engine.

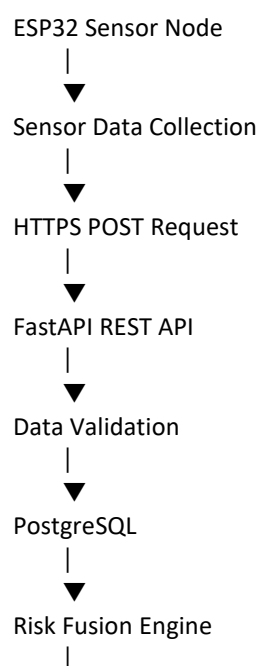
Finally, the processed information is presented through a React dashboard that provides real-time visualization of sensor values, calculated risk scores, and historical records.

The modular architecture allows easy migration from simulation to physical ESP32 hardware.

System Architecture



Data Flow





React Dashboard

Circuit & Sensors

The Zone Node is implemented using an ESP32 microcontroller simulated in Wokwi.

The node collects environmental information from multiple virtual sensors representing temperature, humidity, smoke, gas, water level, vibration, flame, occupancy, and battery voltage.

After collecting the measurements, the ESP32 transmits the sensor readings to the backend through HTTPS POST requests.

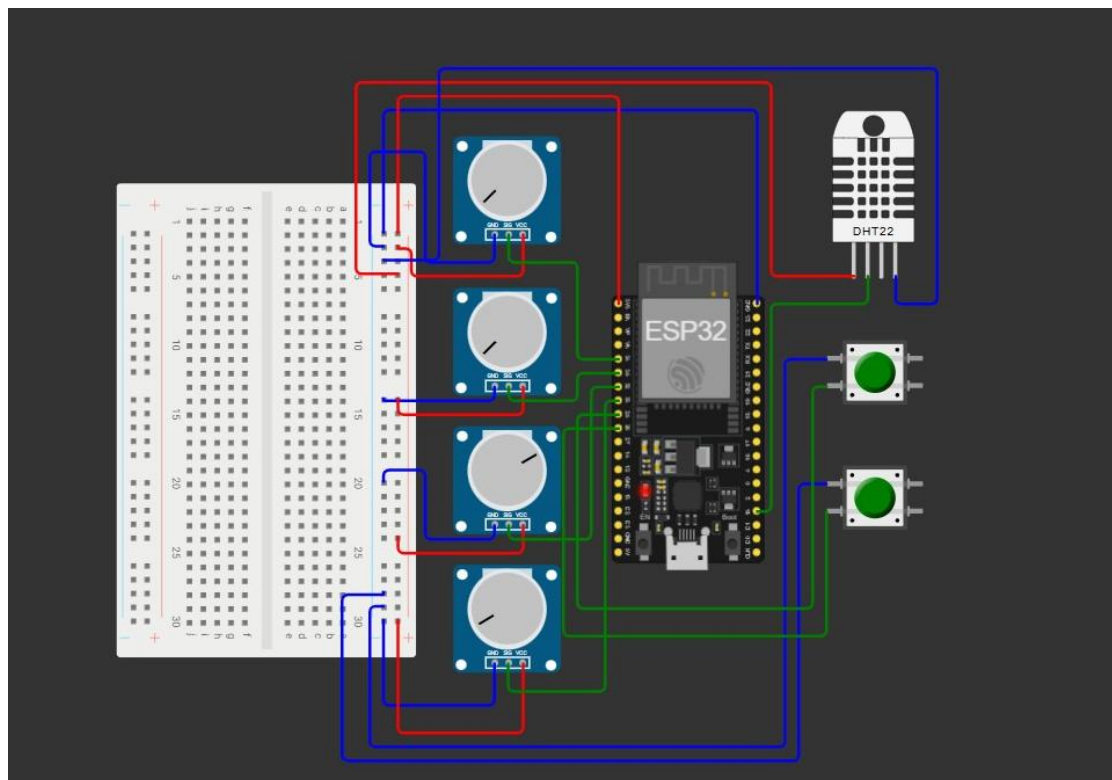
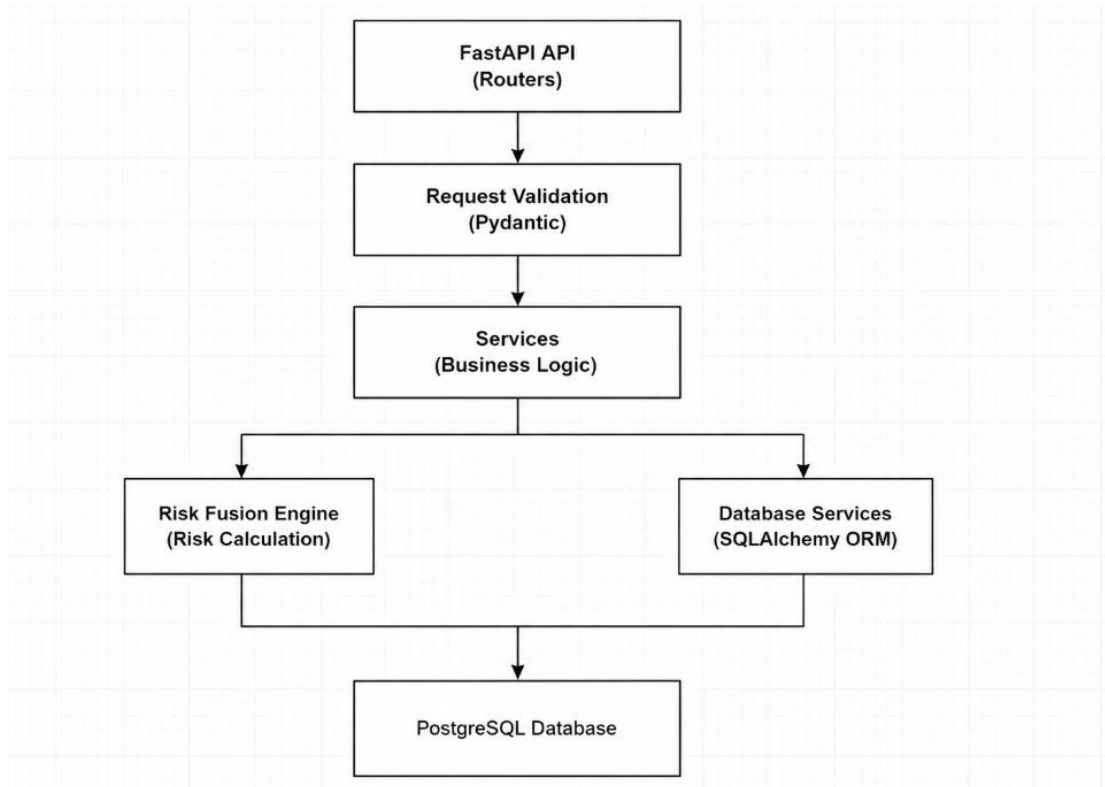


Figure 1: Wokwi Circuit

Backend Architecture



The backend is developed using FastAPI.

The backend performs request validation, stores sensor readings in PostgreSQL, executes the Risk Fusion Engine, generates risk assessments, and exposes REST APIs for frontend visualization.

The modular backend consists of routers, services, database models, schemas, and utility components.

The backend is implemented using FastAPI, SQLAlchemy ORM, PostgreSQL, Alembic, and Pydantic to provide a modular, secure, and maintainable server-side architecture.

Database

PostgreSQL is used as the primary database.

The database stores sensor readings, generated risk assessments, and zone information.

SQLAlchemy ORM is used to simplify database interaction while reducing SQL injection risks.

The database design allows efficient retrieval of historical sensor records for visualization and analysis.

[illegible]

Figure 2. Database Record

Technology Stack

Layer	Technology
Backend	FastAPI
Database	PostgreSQL
ORM	SQLAlchemy
Frontend	React + TypeScript
IoT	ESP32 (Wokwi)
Communication	HTTPS REST API
Tunnel	Cloudflare Tunnel

REST APIs

The backend exposes REST APIs for receiving sensor readings and retrieving calculated risk assessments.

Implemented APIs include:

- ✓ /api/v1/zones
- ✓ /api/v1/sensor-readings
- ✓ /api/v1/risk/preview

- ✓ /api/v1/risk-assessments
- ✓ /api/v1/zones/{id}/risk-assessments
- ✓ /api/v1/zones/{id}/latest-risk

FastAPI automatically generates interactive Swagger documentation for testing and verification.

The screenshot displays the Swagger UI for the RoboFusion SCS-RG API. The header includes the title 'RoboFusion SCS-RG API' with version '0.1.0' and 'OAS 3.1' tags, and a link to the OpenAPI JSON file. Below the header, the 'RoboFusion Smart Campus Safety & Response Grid (SCS-RG)' is introduced as an IoT-based platform developed for the RoboFusion Techathon 2026. The 'Core Features' section lists: Campus Zone Management, IoT Sensor Data Collection, AI-Based Risk Assessment Engine, Historical Risk Analytics, and Backend & Database Health Monitoring. The 'Team' section identifies 'GenZ Ignite' with members Md. Esat Jahan Akhi and Md. Muniruzzaman Bony. Technical details mention the use of FastAPI, PostgreSQL, SQLAlchemy, and Pydantic v2, along with contact information for GenZ Ignite and a MIT License. The main content area is organized into sections: 'Zones' (listing GET endpoints for listing and retrieving zones), 'Risk Assessments' (listing GET endpoints for listing assessments, retrieving by zone, latest risk, and by assessment ID), 'Sensor Readings' (listing a POST endpoint for creating readings and two GET endpoints for listing and retrieving readings), 'Risk' (listing a POST endpoint for previewing fixed risk), and 'System' (listing GET endpoints for API root, health check, and database health check).

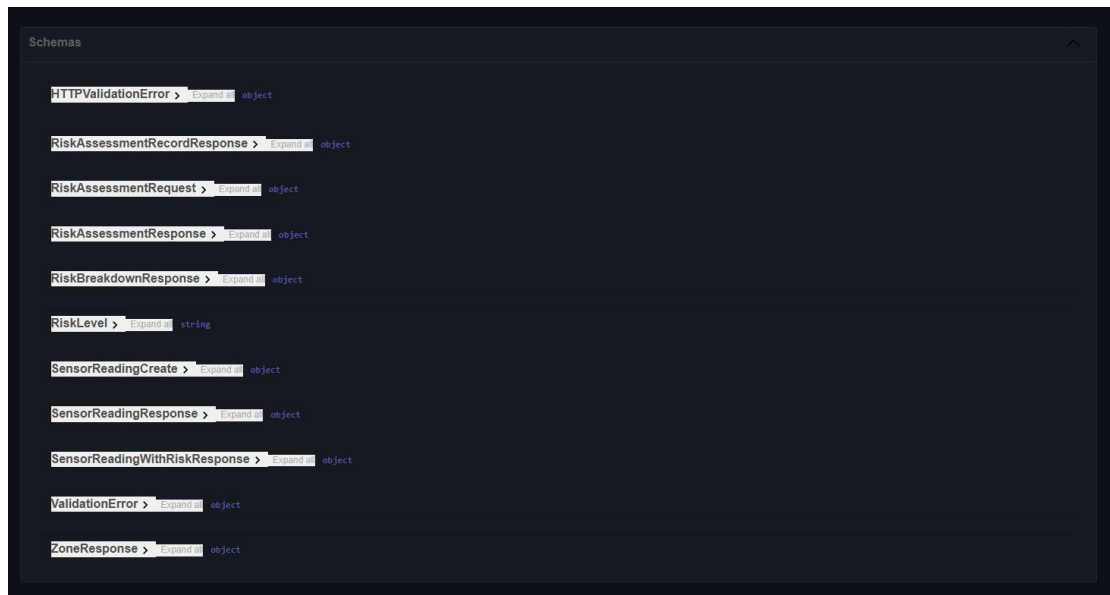


Figure 3. Swagger API

Risk Fusion Engine

The Risk Fusion Engine is responsible for evaluating the overall safety condition of a monitored zone. It converts each sensor reading into a normalized risk value ranging from **0 to 100**, applies predefined weights to each sensor category, and calculates a single weighted risk score. Finally, emergency override rules are evaluated to ensure that critical hazards are never underestimated.

Sensor Thresholds

Each sensor has a predefined **Safe Limit** and **Critical Limit**. Sensor readings are normalized into a risk value between **0 and 100** based on these thresholds.

Sensor	Safe Limit	Critical Limit	Risk Direction
Temperature	30 °C	60 °C	Increasing
Humidity	70 %	100 %	Increasing
Smoke	30 ppm	300 ppm	Increasing
Gas	50 ppm	500 ppm	Increasing
Water Level	10 %	100 %	Increasing
Vibration	0.2	3.0	Increasing

Sensor	Safe Limit	Critical Limit	Risk Direction
Flame	Not Detected	Detected	Binary
Occupancy	20 Persons	100 Persons	Increasing
Battery Voltage	4.5 V	3.3 V	Decreasing

For the flame sensor, the risk is assigned directly:

- ✓ Flame Detected → Risk Score = 100
- ✓ Flame Not Detected → Risk Score = 0

Risk Weights

After normalization, every sensor contributes to the final risk score according to the following weights.

Risk Category	Weight	Contribution
Temperature	0.15	15 %
Humidity	0.05	5 %
Smoke	0.18	18 %
Gas	0.17	17 %
Water	0.12	12 %
Vibration	0.10	10 %
Flame	0.13	13 %
Occupancy	0.05	5 %
Battery	0.05	5 %
Total	1.00	100 %

The higher weights assigned to **Smoke** and **Gas** reflect their importance in identifying fire and toxic gas emergencies.

Weighted Risk Calculation

The weighted base risk score is calculated using the following equation:

$$R_{base} = 0.15R_{temperature} + 0.05R_{humidity} + 0.18R_{smoke} + 0.17R_{gas} + 0.12R_{water} + 0.10R_{vibration} + 0.13R_{flame} + 0.05R_{occupancy} + 0.05R_{battery}$$

Since all weights sum to **1.00**, the weighted score remains within the range of **0–100** before emergency override rules are applied.

Emergency Override Rules

Although the weighted score provides the overall safety condition, several emergency situations require immediate action. Therefore, three emergency override rules are implemented.

Override Rule 1 - Flame Emergency

If the flame sensor detects fire, the final risk score is immediately increased to at least **90**, regardless of the weighted score.

Condition

- ✓ Flame Detected = True

Action

Final Score = $\max(\text{Weighted Score}, 90)$

Reason Generated

- ✓ Flame sensor triggered.

Override Rule 2 - Combined Smoke and Gas Emergency

When both smoke and gas risks reach extremely high levels, the system assumes a possible fire or toxic gas emergency.

Conditions

- ✓ Smoke Risk ≥ 80
- ✓ Gas Risk ≥ 80

Action

Final Score = $\max(\text{Weighted Score}, 85)$

Reason Generated

- ✓ Combined smoke and gas emergency condition detected.

Override Rule 3 - Severe Flooding

When the normalized water-level risk reaches 90 or higher, the system classifies the situation as a severe flooding event.

Condition

- ✓ Water Risk ≥ 90

Action

Final Score = max(Weighted Score, 80)

Reason Generated

- ✓ Severe flooding condition detected.

Final Risk Score

After evaluating the weighted score and all emergency override rules, the final score is restricted to the range of **0–100** and rounded to two decimal places.

$$R_{final} = Clamp(R_{adjusted}, 0, 100)$$

Risk Level Classification

The final numeric score is converted into one of four operational risk levels.

Final Score	Risk Level	Description
Less than 25	LOW	Normal operating condition
25 – 49.99	MEDIUM	Attention recommended
50 – 74.99	HIGH	Immediate monitoring required
75 – 100	CRITICAL	Emergency condition

The backend uses the following classification logic:

- Score $\geq 75 \rightarrow$ **CRITICAL**
- Score $\geq 50 \rightarrow$ **HIGH**
- Score $\geq 25 \rightarrow$ **MEDIUM**
- Otherwise $\rightarrow$ **LOW**

Results

The complete system was validated using Wokwi simulation.

Sensor readings were successfully transmitted to the backend using HTTPS.

The backend stored the data in PostgreSQL, generated risk assessments, and displayed the results through the React dashboard.

Both LOW and CRITICAL operating conditions were successfully demonstrated.

```
Device ID: MOKMI-IOT-LAB-001
Zone ID: 1
Temperature: 24.00 C
Humidity: 40.00 %
Smoke raw ADC: 0
Smoke: 0.00 ppm
Gas raw ADC: 0
Gas: 0.00 ppm
Water raw ADC: 2958
Water level: 72.23 %
Vibration raw ADC: 212
Vibration level: 0.16
Flame status: CLEAR
Motion status: CLEAR
Occupancy count: 0
Battery voltage: 4.90 V

JSON PAYLOAD:
{"zone_id":1,"device_id":"MOKMI-IOT-LAB-001","temperature_c":24,"humidity_percent":40,"smoke_ppm":0,"gas_ppm":0,"water_level_percent":72.23,"vibration_level":0.16,"flame_detected":false,"motion_detected":false,"occupancy_count":0,"battery_voltage":4.9,"captured_at":null}

Sending POST request to: https://soil-nit-tiny-hampton.trycloudflare.com/api/v1/sensor-readings
```

Figure 4. LOW Serial Output

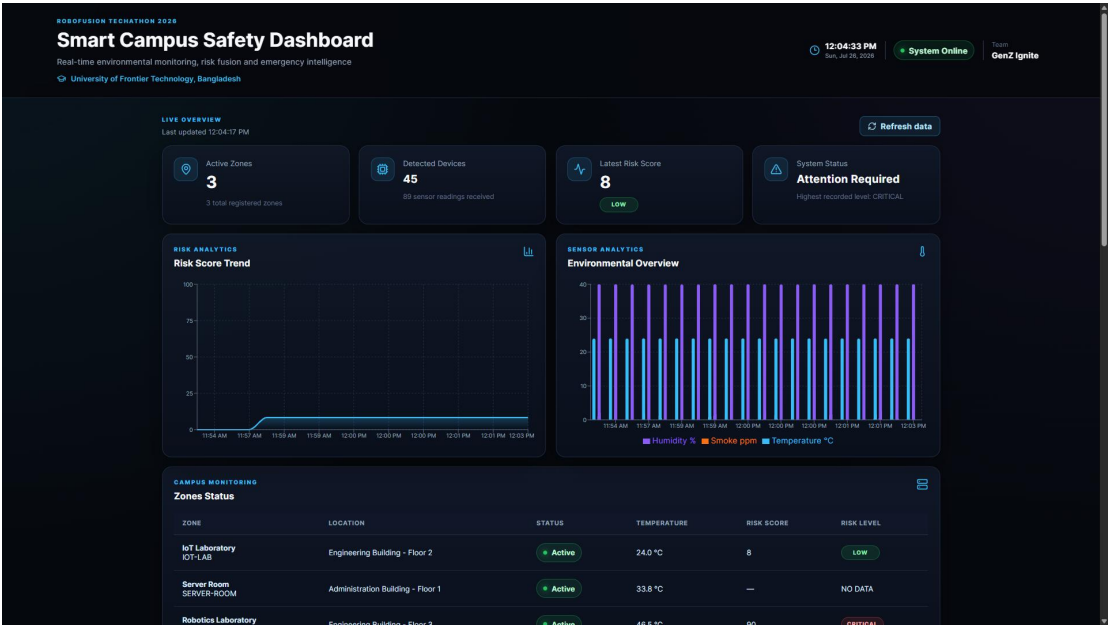




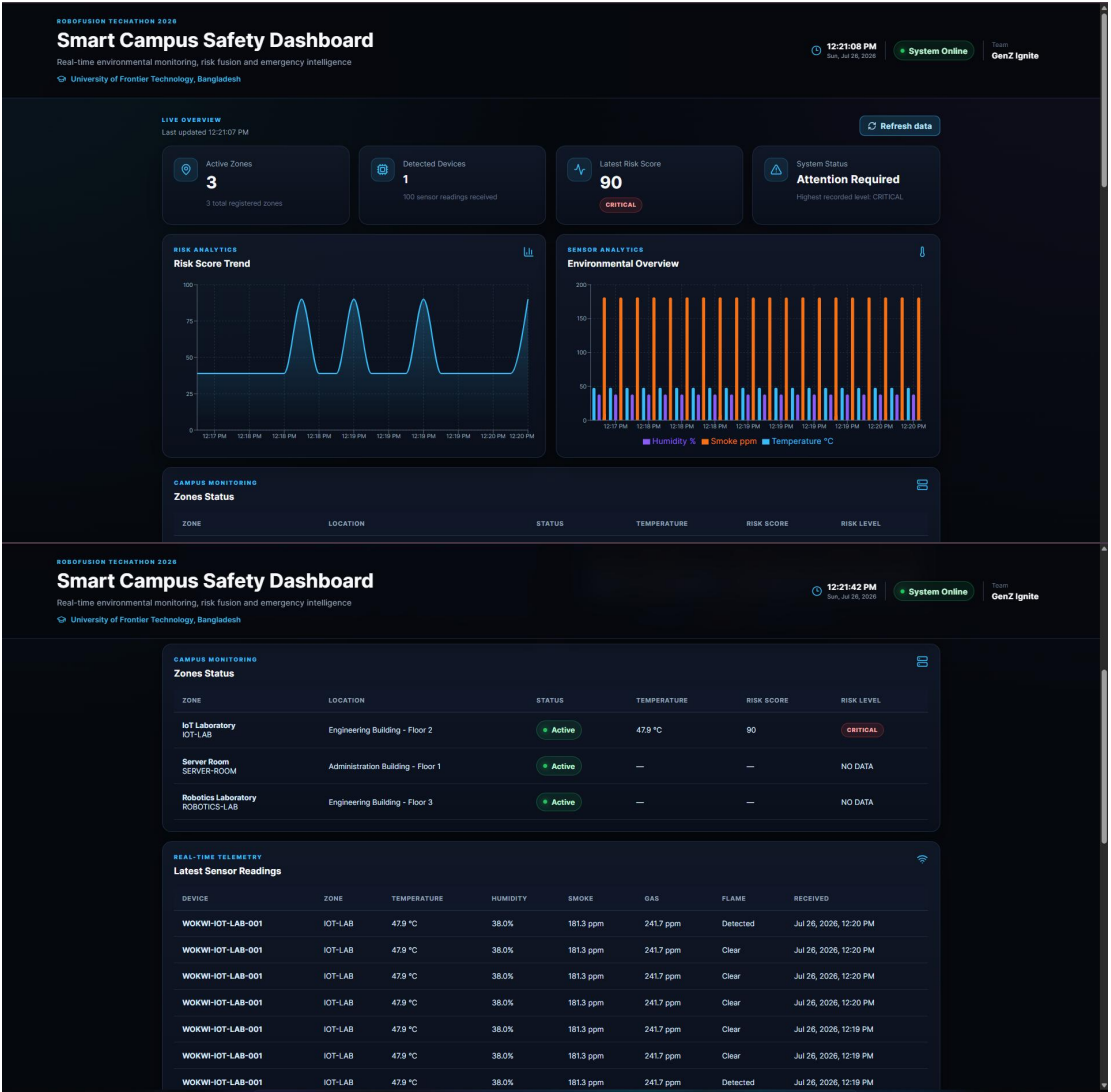
Figure 5. LOW Dashboard

```
Device ID: WOKWI-IOT-LAB-001
Zone ID: 1
Temperature: 47.90 C
Humidity: 38.00 %
Smoke raw ADC: 2474
Smoke: 181.25 ppm
Gas raw ADC: 2474
Gas: 241.66 ppm
Water raw ADC: 2518
Water level: 61.49 %
Vibration raw ADC: 2502
Vibration level: 1.83
Flame status: DETECTED
Motion status: CLEAR
Occupancy count: 0
Battery voltage: 4.90 V

JSON PAYLOAD:
{"zone_id":1,"device_id":"WOKWI-IOT-LAB-001","temperature_c":47.9,"humidity_percent":38,"smoke_ppm":181.25,"gas_ppm":241.66,"water_level_percent":61.49,"vibration_level":1.83,"flame_detected":true,"motion_detected":false,"occupancy_count":0,"battery_voltage":4.9,"captured_at":null}

Sending POST request to: https://soil-mil-tiny-hampton.trycloudflare.com/api/v1/sensor-readings
```

Figure 6. CRITICAL Serial Output



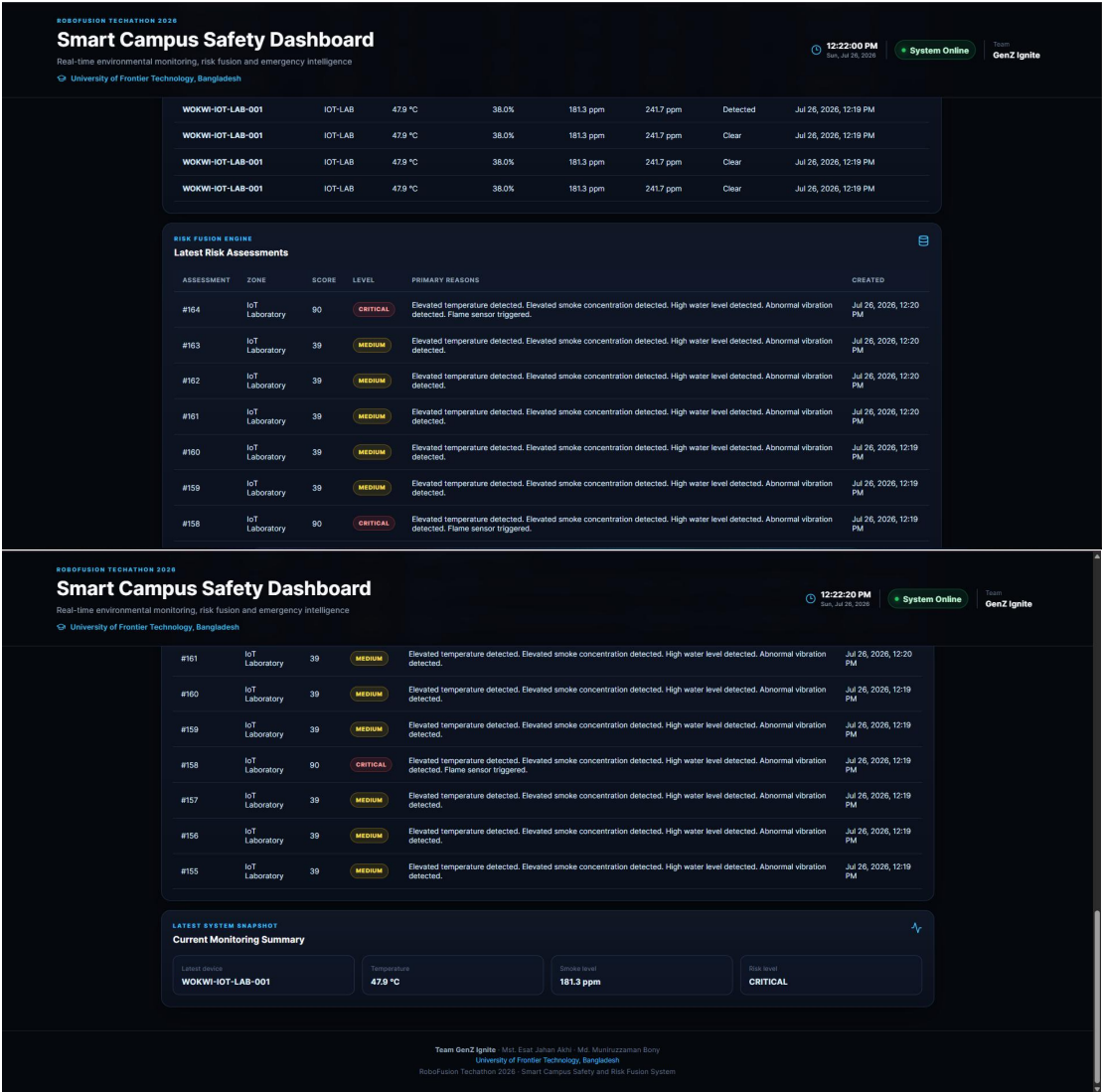


Figure 7. CRITICAL Dashboard

Validation Table

Component	Status
ESP32 Simulation	✓
HTTPS Communication	✓
FastAPI Backend	✓
PostgreSQL	✓
Risk Fusion Engine	✓
React Dashboard	✓

Security

The system incorporates several software engineering best practices.

- ✓ HTTPS communication through Cloudflare Tunnel
- ✓ Request validation using Pydantic
- ✓ SQLAlchemy ORM
- ✓ Environment variable configuration
- ✓ Health monitoring endpoints
- ✓ Modular backend architecture

Limitations

The current prototype uses Wokwi simulation instead of physical hardware.

Cloudflare Quick Tunnel provides temporary public URLs.

The current implementation focuses on a single simulated zone and does not include authentication or notification services.

These limitations do not affect the demonstration of the proposed architecture.

Future Improvements

Future enhancements include:

- ✓ Physical ESP32 deployment
- ✓ Multiple campus zones
- ✓ MQTT communication
- ✓ LoRa connectivity
- ✓ SMS and Email alerts
- ✓ AI-based anomaly detection
- ✓ Mobile application
- ✓ Cloud deployment

Conclusion

The Smart Campus Safety & Response Grid successfully demonstrates an end-to-end IoT safety monitoring platform.

The system integrates ESP32 simulation, FastAPI, PostgreSQL, a weighted Risk Fusion Engine, and a React dashboard into a unified architecture capable of real-time environmental monitoring and intelligent risk assessment.

The modular design allows straightforward migration from simulation to real hardware, making the proposed solution suitable for future smart campus deployments.

References

1. FastAPI Documentation
<https://fastapi.tiangolo.com/>
2. React Documentation
<https://react.dev/>
3. PostgreSQL Documentation
<https://www.postgresql.org/docs/>
4. SQLAlchemy Documentation
<https://docs.sqlalchemy.org/>
5. Wokwi Documentation
<https://docs.wokwi.com/>
6. Cloudflare Tunnel Documentation
<https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/>
7. ESP32 Technical Reference Manual
<https://www.espressif.com/en/support/documents/technical-documents>